International Journal of Control, Automation, and Systems (2026) 24:666–677
https://doi.org/10.1007/s12555-026-00038-1

REGULAR PAPER

Efficient Learning-Based Path Planning on Receding Horizon with Conditional Flow Matching

Yongjae Lim[1] · Yeonsoo Park[2] · Jeongtae Huh[3] · Youngsoo Han[4] · H. Jin Kim[4]

Received: 4 June 2025 / Revised: 31 October 2025 / Accepted: 6 January 2026 / Published online: 11 March 2026
© The Author(s) 2026

Abstract

Denoising-based planning, which gradually refines noisy initial trajectories, has shown strong performance in generating safe and optimized ones. However, such approaches often suffer from slow convergence to an optimal trajectory, which can be problematic in real-world applications where computational delays may lead to planning failures or unsafe behavior. In this paper, we propose a novel learning-based path-planning method that leverages a normalizing flow framework to adaptively adjust the trajectory length, thereby enabling the planner to operate within a given computational time budget. To achieve this, our network refines a single step in the trajectory at each iteration, sequentially progressing along the planning horizon, rather than refining the entire trajectory as in diffusion-based methods. Consequently, our proposed method improves the flexibility of path planning by allowing the receding horizon length to be dynamically adjusted according to computational constraints. Furthermore, the inference results of the network can serve as initial guidance within the model predictive path integral framework, thereby reducing the number of optimization iterations required for path convergence and improving overall optimization efficiency. We validate the efficiency of our method through comparisons with diffusion-like approaches in simulation and demonstrate its practicality through hardware experiments.

Keywords Path planning · Learning-based path planning · Control · Generative model · Conditional flow matching

1 Introduction

Path planning for autonomous systems focuses on generating a safe path that guides the system to its goal without collisions. Traditional path-planning approaches, such as model predictive control (MPC), are capable of producing optimal paths using predefined cost functions and constraints [1–6]. However, as the complexity of the environment increases, traditional path-planning approaches become less practical due to their high computational cost.

Sampling-based variational inference approaches [7–11] maintain reasonable computational efficiency even as the complexity of the environment and system increases. They compute the optimal control input sequence by reformulating a stochastic optimal problem (SOP) as an inference problem [7, 12, 13], leveraging the relationship between the defined cost function and the measured optimality of the sampled trajectories. However, the performance of this approach heavily relies on the quality of the sampled paths. If only a few sampled paths meet the defined constraints, the resulting path quality can degrade significantly. Various techniques have been proposed to constrain sampling to regions that meet the

Recommended by Guest Editor Jungwon Yoon under the direction of Senior Editor Han-Lim Choi.

✉ H. Jin Kim
hjinkim@snu.ac.kr

Yongjae Lim
yongjae.lim@snu.ac.kr

Yeonsoo Park
ys_park@snu.ac.kr

Jeongtae Huh
49ball@snu.ac.kr

Youngsoo Han
cat7945@snu.ac.kr

[1] Department of Mechanical Engineering, Seoul National University, Seoul 08826, South Korea

[2] Interdisciplinary Program in AI, Seoul National University, Seoul 08826, South Korea

[3] Department of Future Automotive Mobility, Seoul National University, Seoul 08826, South Korea

[4] Department of Aerospace Engineering, Seoul National University, Seoul 08826, South Korea

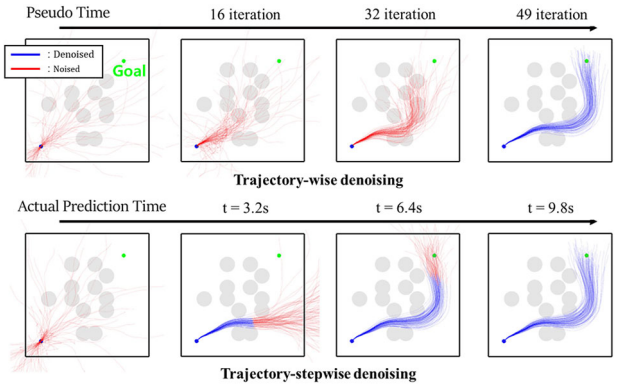


Fig. 1 The trajectory-wise denoising process shown at the top removes noise from the entire trajectory sequentially, while the proposed stepwise denoising process shown at the bottom denoises each specific step of the trajectory individually. The blue curves can be used for planning, whereas the red ones cannot

specified constraints [8, 9, 14], but these methods rely on additional modules for constraint satisfaction, making them computationally slow. To accelerate the inference of control input distributions, learning-based approaches provide initial guidance for exploration [15, 16]. In contrast, previous studies have shown that the performance of networks tends to degrade when the input environment differs from the training data.

Recently, denoising-based methods, such as diffusion and flow matching (FM), have been applied to path-planning networks [17–19], demonstrating strong performance in generating safe and optimized trajectories, even in environments not seen during training. In this context, denoising refers to the process of progressively refining a noisy trajectory toward an optimal one. For example, [19] employs FM to generate control input sequences that connect the start and goal positions by iteratively refining the entire trajectory until convergence. We refer to this as *trajectory-wise denoising*, in contrast to *trajectory-stepwise denoising*, which refines each step individually per iteration, as shown in Fig. 1. However, trajectory-wise denoising requires iterative inference over a fixed number of denoising steps, which can be inefficient in practice. Consequently, the intermediate results cannot be used, since they are still noisy trajectories, leading to inefficiency, particularly when fast planning is necessary.

In this paper, we propose a novel learning-based path-planning method, utilizing a new trajectory-stepwise denoising process by defining normalizing flow (NF) framework using conditional flow matching (CFM) [20]. Unlike previous approaches, our method allows the use of intermediate network results. This is possible, because our approach sequentially denoises a specific noisy step in the trajectory rather than removing noise from the entire trajectory. This stepwise denoising aligns with the actual prediction time horizon (*receding horizon*), enabling faster and more flexible inference by adjusting the number of steps based on the desired prediction horizon length. In addition, we can use the network outputs as the initial guidance for model predictive path integral (MPPI) framework, represented by trajectory-wise denoising, to refine the network's results while still achieving quick convergence. To validate our approach, we demonstrate the efficiency of our work through comparisons with other trajectory-wise denoising methods [7, 21] in simulation (e.g., planning time and success rate). Additionally, we establish its practical applicability through scenarios that require escaping local minima, which cannot be resolved without considering a long horizon, as well as scenarios that demand rapid path planning.

The contributions of this work are summarized:

1. We define NF framework to enable the trajectory-stepwise denoising rather than trajectory-wise denoising. By aligning denoising steps with a receding horizon, our method allows dynamic adjustment of trajectory length according to available computational resources, improving planning flexibility.
2. The inference results of the network can be used as the initial guidance for the trajectory-wise denoising method, significantly reducing the number of denoising iterations required for trajectory convergence.
3. We validate our method through simulation comparisons and further demonstrate its applicability through hardware experiments, highlighting its capability to avoid local minima and enable fast planning.

2 Method

In this section, we explain how the proposed network is trained to enable trajectory stepwise denoising. We first introduce NF framework and CFM, and then describe how our denoising-based planning approach leverages this framework to realize stepwise trajectory refinement.

2.1 Explanation of Conditional Flow Matching

NF framework is used in generative models for creating desired samples from a complex distribution. The generative models with NF framework learn an invertible probability density transformation, enabling the mapping from a simple distribution to a complex one. FM is a method for training NF-based generative models and estimates a time-dependent vector field to progressively transform an initial distribution p_0 into a target distribution p_1, where p_t is the intermediate distribution at $t \in [0, 1]$. The objective function for FM is formulated as

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}_{t\sim\mathcal{U}[0,1],x\sim p_t(x)} \| v_\theta(x) - u_t(x) \|^2,$$

where $v_\theta(x)$ is the learned vector field and $u_t(x)$ is the ground-truth vector field. Since $u_t(x)$ is often intractable, CFM modifies the objective by incorporating conditional dependencies, where $x_1 \sim q_s(x_1)$ serves as a conditioning variable sampled from the target distribution p_1 [20]

$$L_{\mathrm{CFM}}(\theta) = \mathbb{E}_{t,x,x_1} \| v_\theta(x) - u_t(x \mid x_1) \|^2; \tag{1}$$

(1) maintains the optimization structure of FM while improving computational feasibility [20]. In addition, (1) efficiently models structured probability flows, making it well suited for tasks requiring controlled sampling.

2.2 Definition of Optimal Trajectory Distribution

The control input u_t at time t following an initial random policy $p(u_t)$ is usually an unbiased Gaussian distribution to facilitate exploration. Using an unbiased Gaussian distribution for exploration has the advantage of enabling the agent to explore all possible regions (*completeness*), allowing it to discover the optimal trajectory based on the defined cost function (*optimality*). However, this approach requires a large number of samples to identify an optimal solution, making it inefficient. To address this, we leverage the relationship between the defined cost function and the measured optimality of the sampled trajectories to guide the exploration more efficiently [13]

$$p(o = 1|\tau) \propto \exp\left(-\frac{J(\tau)}{\lambda}\right),$$

where λ is the inverse temperature, and o is the binary indicator variable representing whether the sampled trajectory τ is optimal and $J(\tau)$ is the cost function of the sampled trajectory. The optimal trajectory distribution, $p^*(\tau)$, is defined as follows:

$$p^*(\tau) = p(\tau|o = 1) = \frac{1}{\eta} p(o = 1|\tau)\, p(\tau), \tag{2}$$

where η is the normalizing constant and $p(\tau)$ represents the prior trajectory distribution.

2.3 Optimal Policy in Environment Context

We extend (2) with the surrounding environment E

$$p^*(\tau|E) = p(\tau|o = 1, E) = \frac{1}{\eta} p(o = 1|\tau, E)\, p(\tau). \tag{3}$$

We show that the optimal policy, $p^*(u_i|E)$, satisfies

$$p^*(u_i|E) = \frac{1}{\eta_i} H(u_i \mid \tau_{0:i},\ E)\, p(u_i), \tag{4}$$

where $\eta_i = \int H(u_i \mid \tau_{0:i},\ E)\, p(u_i)\, \mathrm{d}u_i$, $\eta = \prod_{i=0}^{K-1} \eta_i$, H represents the effect of the surrounding environment E on $p(\tau)$, and K is the length of τ. Furthermore, u_i is the control input at step i, $\tau_{0:j} = \{U_{0:j-1}; x_0\}$, $U_{0:j-1} = \{u_0, u_1, \ldots, u_{j-1}\}$, and x_0 represents the initial state. We can decompose $p(o = 1, U_{0:k-1} \mid x_0, E)$ as follows:

$$\begin{aligned} p(o = 1, U_{0:k-1} \mid x_0, E) &= \frac{p(o = 1, U_{0:k-1}, x_0, E)}{p(x_0, E)} \\ &= \frac{p(o = 1, U_{0:k-1}, x_0, E)}{p(U_{0:k-1}, x_0, E)} \cdots \frac{p(u_0, x_0, E)}{p(x_0, E)} \\ &= p(o = 1 \mid \tau_{0:k}, E) \prod_{i=0}^{k-1} p(u_i), \end{aligned} \tag{5}$$

where $p\left(u_j \mid \tau_{0:j-1}, E\right) = p\left(u_j\right)$ and $p(o = 1, U_{0:k-1} \mid x_0, E)$ can be decomposed into optimal step policies.

$$\begin{aligned} p(o = 1, U_{0:k-1} \mid x_0, E) &= \frac{p(o = 1, U_{0:k-1}, x_0, E)}{p(x_0, E)} \\ &= \frac{p(o = 1, U_{0:k-1}, x_0, E)}{p(o = 1, U_{0:k-1}, x_0, E)} \cdots \frac{p(o = 1, x_0, E)}{p(x_0, E)} \\ &= \prod_{i=0}^{k-1} p(u_i \mid o = 1, U_{0:i-1}, x_0, E), \end{aligned} \tag{6}$$

where $U_{0:-1}$ is an empty set. From (5) and (6), we can get

$$\begin{aligned} p(o = 1 \mid \tau_{0:k}, E) &= \prod_{i=0}^{k-1} \frac{p(u_i \mid o = 1, U_{0:i-1}, x_0, E)}{p(u_i)} \\ &= \prod_{i=0}^{k-1} H(u_i \mid \tau_{0:i},\ E). \end{aligned}$$

Therefore, the optimal trajectory distribution in (3) is redefined as follows:

$$\begin{aligned} p^*(\tau_{0:k}|E) &= \frac{1}{\eta} p(o = 1|\tau_{0:k}, E)\, p(\tau_{0:k}) \\ &= \prod_{i=0}^{k-1} \frac{1}{\eta_i} H(u_i \mid \tau_{0:i},\ E)\, p(u_i). \end{aligned} \tag{7}$$

Using $p^*(\tau_{0:k}|E) = \prod_{i=0}^{k-1} p^*(u_i|E)$, (4) holds. The optimal behavior policy depends on the random policy $p(u_i)$ and the effect of the surrounding environment E. In this paper, we model $p(u_i)$ as an unbiased Gaussian.

2.4 Influence of the Optimal Policy at Each Step Toward the Terminal State

Each action along the trajectory influences the terminal state by altering the intermediate system states, which subsequently propagate through system dynamics. To model the

impact of an action at each step toward the terminal state, we define the k-step instant terminal state.

Definition 1 (*k-step instant terminal state)* Let N denote the horizon length. If the first k steps are executed according to the optimal policy $p^*(u_i)$, and the remaining $(N-k)$ steps are taken under the random policy $p(u_i)$, then the resulting state after N steps is referred to as the k-step instant terminal state $x^*_{k,N}$. In continuous time, $x^*_{k,N}$ is rewritten as $x^*_{t,T}$, where T represents the terminal time.

We model the difference between the k-step instant terminal state and the $(k+1)$-step instant terminal state as the effect of H. When the optimal state x^*_i is given, the next optimal state x^*_{i+1} can be defined from the i-step optimal control input u^*_i and system dynamics as follows:

$$x^*_{i+1} = f_{sys}\left(x^*_i, u^*_i\right), \quad u^*_i \sim (4).$$

The $(i+1)$-step instant terminal state is defined as follows:

$$\begin{aligned} x^*_{i+1,N} &= h\left(x^*_{i+1}, U_{i+1:N-1}\right) \\ &= h\left(f_{sys}\left(x^*_i, u^*_i\right), U_{i+1:N-1}\right), \end{aligned} \tag{8}$$

where N denotes the terminal step, and h is the function that applies f_{sys} iteratively until terminal state. Let us define $v_k(x^*_{k,N}, u_k)$ with time interval Δt as the gap between $x^*_{k+1,N}$ and $x^*_{k,N}$. From the Taylor expansion of (8), it is approximated as follows:

$$\begin{aligned} x^*_{k+1,N} - x^*_{k,N} &= v_k\left(x^*_{k,N}, u_k\right)\Delta t \\ &= h\left(f_{sys}\left(x^*_k, u^*_k\right), U_{k+1:N-1}\right) \\ &\quad - h\left(f_{sys}\left(x^*_k, u_k\right), U_{k+1:N-1}\right) \\ &\simeq \frac{\partial h}{\partial f_{sys}}\left(f_{sys}\left(x^*_k, u^*_k\right) - f_{sys}\left(x^*_k, u_k\right)\right) \\ &\simeq \frac{\partial h}{\partial f_{sys}}\frac{\partial f_{sys}}{\partial u_k}\left(u^*_k - u_k\right). \end{aligned} \tag{9}$$

We note that (9) holds when f_{sys} is differentiable and locally smooth around the point of approximation under small perturbations. In the following section, we define the loss function to train the optimal policy (4) through NF framework, which is constructed using (9).

2.5 Denoising Process Aligning with Time Domain

In this paper, denoising refers to transforming the unbiased Gaussian distribution $p(u_t)$ into the optimal policy $q(u_t)$. Solving for H in (4) from the initial step to the terminal step corresponds to transforming $p(\tau)$ into $p^*(\tau)$. To realize this transformation, we introduce a denoising kernel function ϕ_k at step k

$$x^*_{k+1,N} = x^*_{k,N} + v_k\left(x^*_{k,N}, u_k\right)\Delta t = \phi_k\left(x^*_{k,N}\right). \tag{10}$$

The whole process from $x^*_{0,N}$ to x^*_N can be represented as follows:

$$x^*_N = \phi_{N-1} \circ \phi_{N-2} \cdots \phi_0\left(x^*_{0,N}\right),$$

where x^*_N is the terminal state sampled by the optimal control input sequence. Using (10), x^*_N can be reformulated as follows:

$$\begin{aligned} x^*_N &= \phi_{N-1}\left(x^*_{N-1,N}\right) \\ &= \phi_{N-2}\left(x^*_{N-2,N}\right) + v_{N-1}\left(x^*_{N-1,N}, u_{N-1}\right)\Delta t \\ &\quad \cdots \\ &= x^*_{0,N} + \sum_{i=0}^{N-1} v_i\left(x^*_{i,N}, u_i\right)\Delta t. \end{aligned}$$

In continuous time, it is written as follows:

$$x^*_T = x^*_{0,T} + \int_0^T v_s\left(x^*_{s,T}, u_s\right) ds. \tag{11}$$

The difference between the k-step instant terminal state's probability density function (PDF) and the $(k+1)$-step instant terminal state's PDF can be modeled as the effect of H. From the Fokker–Planck equation about (11), we obtain

$$\frac{\partial p_t\left(x^*_{t,T}\right)}{\partial t} = -\nabla \cdot \left(v_t\left(x^*_{t,T}, u_t\right) p_t\left(x^*_{t,T}\right)\right). \tag{12}$$

Here, p_t is the probability density of $x^*_{t,T}$ and it is induced by v_t [20]. We can rewrite (12) in the log space as follows:

$$\frac{\mathrm{d}}{\mathrm{d}t}\log p_t\left(x^*_{t,T}\right) = -\left(\nabla \cdot v_t\right)\left(x^*_{t,T}\right). \tag{13}$$

The log density of (13) is derived as follows:

$$\log p\left(x^*_T\right) = \log p_0\left(x^*_{0,T}\right) - \int_0^T \left(\nabla \cdot v_s\right)\left(x^*_{s,T}\right) \mathrm{d}s. \tag{14}$$

Note that $p(x^*_{0,T}) = p(\tau) = P(U)$ and $p(x^*_T) = p^*(\tau)$. (14) means that finding the denoising path, $\int_0^T (\nabla \cdot v_s)\left(x^*_{s,T}\right) \mathrm{d}s$, corresponds to determining the optimal control input sequence $Q(U)$, as shown in Fig. 2. Therefore, by the definition of H in (4), H at time t can be represented by the influence of the vector field v_t in (14).

To estimate H at time t, we approximate the ground-truth vector field v_t with the learnable field $v_{t,\theta}$, parameterized

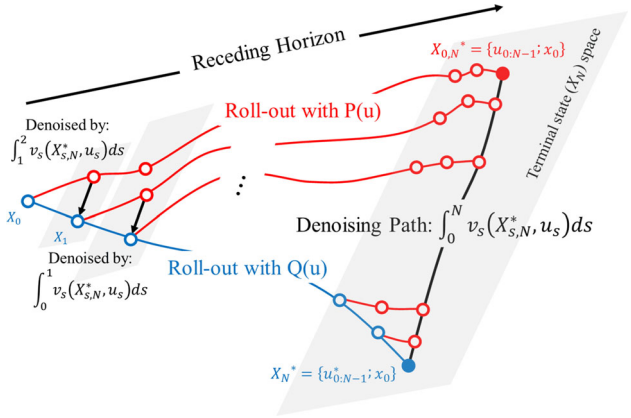


Fig. 2 The denoising path of the terminal state consists of the vector field between the k-step and $(k+1)$-step instant terminal state, where $k \in [0, N-1]$. Each vector field affects the corresponding step by denoising the noisy step to the optimal step. $Q(u)$ denotes the optimal behavior policy $q(u_i)$

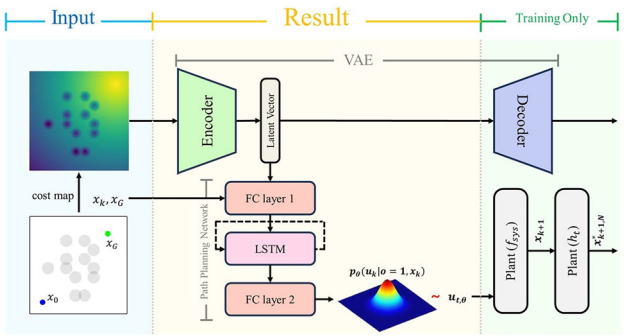


Fig. 3 Architecture of our path-planning network. It receives the cost map, system state, and goal state. The network outputs the control input distribution of the input state's step

by the network weights θ. The network can learn the optimal policy described in (4) by minimizing the following loss function:

$$\mathcal{L}_{FM} = \mathbb{E}\left[||v_{t,\theta}\left(x^*_{t,T}, u^*_t\right) - v_t\left(x^*_{t,T}, u^*_t\right)||^2 \right], \tag{15}$$

where $t \in [0, T]$, $x^*_{t,T} \sim p\left(x^*_{t,T}\right)$, $u^*_t \sim q(u_t)$. $x^*_{t,T}$ can be substituted by $\{U_{t:T-\mathrm{d}t}, U^*_{0:t-\mathrm{d}t}; x_0\}$. We can rewrite $v_{t,\theta}$ as $v_t(x^*_{t,T}, u^*_{t,\theta})$, and using (9), we obtain

$$\begin{aligned} v_t(x^*_{t,T}, u^*_{t,\theta})\mathrm{d}t &= h_t(f_{\mathrm{sys}}(x^*_{t,T}, u^*_{t,\theta}), U_{t+\mathrm{d}t:T-\mathrm{d}t}) - x^*_{t,T} \\ &\simeq \frac{\partial h}{\partial f_{\mathrm{sys}}} \frac{\partial f_{\mathrm{sys}}}{\partial u_t}\left(u^*_{t,\theta} - u_t\right) - x^*_{t,T}, \end{aligned}$$

where $u^*_{t,\theta} \sim p^*_\theta(u_t)$, and $p^*_\theta(u_t)$ denotes the learned distribution of $q(u_t)$ parameterized by the network. By the definition of v_t in (9), $\mathcal{L}_{\mathrm{FM}}$ is rewritten as follows:

$$\mathcal{L}_{\mathrm{FM}} = \mathbb{E}\left[\left\| \frac{\partial h}{\partial f_{\mathrm{sys}}} \frac{\partial f_{\mathrm{sys}}}{\partial u_t}\left(u^*_{t,\theta} - u^*_t\right) \right\|^2 \right]. \tag{16}$$

Consequently, training based on (15) corresponds to minimizing a weighted MSE between the optimal control input u^*_t and $u_{t,\theta}$. The weight in (16) reflects the relative importance of each action along the trajectory.

3 Implementation Detail

In this work, we use the simple kinematics model

$$\dot{x} = v\cos(\psi), \quad \dot{y} = v\sin(\psi), \quad \dot{v} = a, \quad \dot{\psi} = r, \tag{17}$$

where x and y are position, v is the velocity of vehicle, and ψ is the orientation. a and r represent the acceleration and yaw rate, respectively. The state and control input at time t are defined as $x_t = [x, y, v, \psi]^T$ and $u_t = [a, r]^T$, respectively. We discretize (17) with the time interval $\Delta t = 0.2$ s to reduce the receding horizon length needed to generate a path between the start and goal states.

3.1 Environment and Cost Function for Training

We randomly place 12 obstacles and the start point on the 8×8 m map. The obstacles are modeled as circles with a radius of 0.5 m. We let x_{obs} denote the obstacle state in the obstacle set $\mathcal{O}$, x_k the system state at the step k, and x_G the goal state. In this study, the dataset contains only trajectories planned in maps with static obstacles. However, if the dataset includes paths connecting start and goal points in maps with moving objects, we expect that the proposed method can still be effectively applied to scenarios involving dynamic obstacles. To generate the paths in the dataset, we design a cost function $J(\tau)$ for the trajectory τ, which is defined as follows:

$$J(\tau) = \sum_{i=1}^{N} d_G(x_i) + \sum_{i=1}^{N} 1000 D(x_i), \tag{18}$$

where N is the horizon length, $d_G(x_i) = ||x_i - x_G||_2$ is a distance to goal function, and D is an indicator function which is 1 if x_i is in a collision or out of the map and 0 otherwise. The collision is judged if $||x_i - x_{\mathrm{obs}}||_2^2 < (d_s + r_{\mathrm{obs}})^2$, where r_{obs} is the radius of the obstacle and d_s is the safety margin. We set $d_s = 0.1$ m and $N = 50$, and employ an exploring policy $p(u_t) = \mathcal{N}(0, \Sigma_w)$, where $\Sigma_w = \mathrm{diag}\left([0.67^2, 1.0^2]\right)$.

3.2 Loss Function for Training

The whole network architecture is designed as shown in Fig. 3. The architecture details are summarized in Table

Table 1 Layer-wise structure of the FC layer 1, 2, and LSTM in Fig. 3

Block	Main structure	# of layers	Nodes/neurons per layer
State encoder1	Linear → LayerNorm → ReLU → Linear → ReLU	2 FC	$5 \to 16 \to 16$
State encoder2	Linear → LayerNorm → ReLU → Linear → ReLU	2 FC	$5 \to 16 \to 16$
Context net	Linear → ReLU → Linear	2 FC	$(2 \cdot 16 + 128) = 160 \to 48 \to 48$
LSTM	2-layer LSTM (temporal memory)	2 recurrent layers	Input size = output = $50 \times 80 = 4000$ (per layer)
Fc acc mu	Linear → ReLU → Linear → ReLU → Linear → ReLU → Linear → Tanh	4 FC	$4000 \to 400 \to 400 \to 32 \to 1$
Fc dyaw mu	same as *Fc acc mu*	4 FC	$4000 \to 400 \to 400 \to 32 \to 1$
Fc acc dev	Linear → ReLU → Linear → ReLU → Linear → Tanh	3 FC	$4000 \to 400 \to 32 \to 1$
Fc dyaw dev	same as *Fc acc dev*	3 FC	$4000 \to 400 \to 32 \to 1$

1. *State encoder 1* and *2* receive the goal and current states, respectively, and produce their corresponding encoded representations. *context net* takes as input the concatenation of the outputs from both state encoders together with the latent vector obtained from the VAE. *LSTM* processes the output of the *Context net*, sequentially aggregating the contextual features over 50 time steps. The output of *LSTM* is then fed into four fully connected layers (*FC acc mu, FC dyaw mu, FC acc dev, FC dyaw dev*). Finally, the path-planning network outputs the control input distribution at the step of the input state. The training process is described in Algorithm 1. To train the path-planning network with sampled data, we adopt CFM [20] using the loss function in (15). The loss function $\mathcal{L}_{CFM}$ is defined with $u^*_{k,\theta} \sim p_\theta\left(u_k \mid o = 1, x^*_k\right)$

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}\left[\left\| v_k\left(x^*_{k,N}, u^*_{k,\theta}\right) - v_k\left(x^*_{k,N}, u^*_k|\bullet\right)\right\|^2\right], \tag{19}$$

where $\bullet = \{x_0, U^*_{0:k}, U_{k+1:N-1}\}$ and we model the p_θ as an unimodal Gaussian distribution. To use CFM in our problem setup, we should extend CFM to the multi-variable setting, since v_k in (19) is conditioned on $\bullet$. Despite this extension, the theorems introduced in [20] for defining $\mathcal{L}_{\text{CFM}}$ can still be applied directly. (19) can be rewritten using the definition of v_k as follows:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}\left[\left\| x^*_{k+1,N} - \hat{x}^*_{k+1,N}\right\|^2\right],$$

where $x^*_{k+1,N}$ is given by dataset and $\hat{x}^*_{k+1,N}$ is the estimated $(k+1)$-step instant terminal state by substituting k-step optimal control input with $u^*_{k,\theta}$. In addition, to teach the given environment to our network, we employ the variational autoencoder (VAE). We use the normalized cost map of the 64-by-64 environment as the VAE input. The cost at the position (x, y) on the cost map is

$$C(x, y) = \left(\sqrt{x^2_{\text{lim}} + y^2_{\text{lim}}} - d_G\right) - 20(r^c_{\text{obs}} - d^c_{\text{obs}}),$$

where x_{lim}, y_{lim} are the map size, d_G is the distance from the goal, and r^c_{obs} and d^c_{obs} are the radius and distance from the closest obstacle, respectively. Finally, the loss function for training the whole network in Fig. 3 is

$$\mathcal{L} = \mathcal{L}_{\text{CFM}} + \beta\mathcal{L}_{\text{vae}} + \gamma\mathbb{E}\left[\left(\log\left(\det\left(\sqrt{\Sigma_\theta}\right)\right)\right)^2\right], \tag{20}$$

where $\mathcal{L}_{\text{vae}}$ is computed as the MSE between the input and the reconstructed cost map, and $\sigma_\theta = \sqrt{\Sigma_\theta}$ is the network output representing the standard deviation, parameterized by the network weights. We set the $\beta = 0.01$, $\gamma = 0.003$, and the learning rate $\alpha = 0.001$ for the training.

Algorithm 1 Training

μ^i_u, Σ^i_u: i-step mean and covariance of u
μ^i_θ, Σ^i_θ: i-step mean and covariance from the network
load dataset D
while converges loss **do**
  contruct batch by randomly choosing episodes
  for $i = 1 \ldots N-1$
    for $j = 0 \ldots i-1$
      $x_{j+1} = f_{sys}\left(x_j, u^*_j\right)$
    obtain $x^*_{k+1,N}$ by $u^*_i \sim \mathcal{N}\left(\mu^i_u, \Sigma^i_u\right)$
    obtain $\hat{x}^*_{k+1,N}$ by $\hat{u}_i \sim \mathcal{N}\left(\mu^i_\theta, \Sigma^i_\theta\right)$
    calculate $\mathcal{L}$ using (20)
    update network

3.3 How to Construct the Dataset

We construct the ground-truth control input sequence distribution using the top K update rule in TD-MPC [22]. This distribution is calculated by taking the importance-weighted average of the top M trajectories among the whole sampled

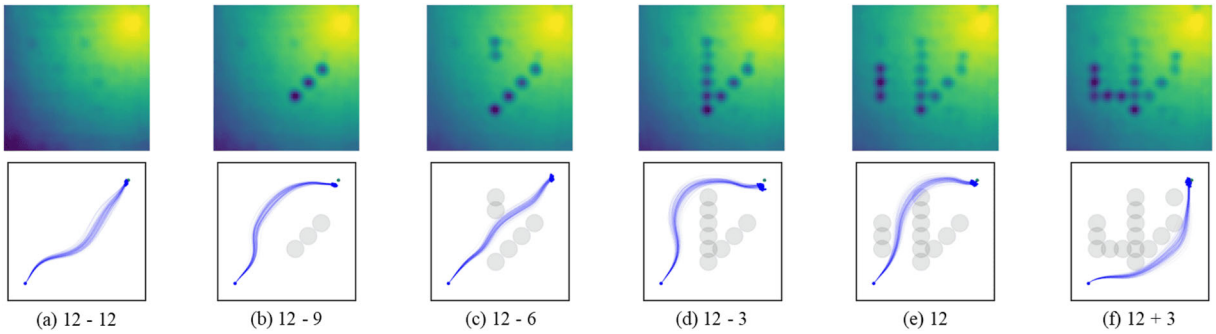


Fig. 4 The inference results of our network. The obstacle is added to impede the agent. We train our network with a dataset of cases with randomly located 12 obstacles. Our network can avoid obstacles and reach the goal point even in cases it has not been explicitly trained on

trajectories evaluated by (18). The sampled trajectories are obtained from the sampled control input sequences and initial state. The update rule for mean μ_u and covariance $\mathbf{\Sigma}_u$ of the control input sequence distribution is defined as follows:

$$\mu_u^j = \frac{\sum_{i=1}^{M} \Omega_i \Gamma_i^*}{\sum_{i=1}^{M} \Omega_i}, \quad \Sigma_u^j = \frac{\sum_{i=1}^{M} \Omega_i \left(\Gamma_i^* - \mu_u^j\right)^2}{\sum_{i=1}^{M} \Omega_i}, \tag{21}$$

where j represents the step, $\Omega_i = \exp(-J(\tau_i))$, and Γ_i^* denotes ith trajectory among the top M trajectories. μ_u^i and Σ_u^i denote the ith control input mean and covariance of $\boldsymbol{\mu}_u$ and $\mathbf{\Sigma}_u$, respectively. The iterations terminate when the endpoint of the rollout trajectory, generated using μ_u from the initial state, is within 0.6 m of the goal. Algorithm 2 details the dataset construction.

4 Simulation and Experiment Result

The inference results of our network are illustrated in Fig. 4. Using the trained network output, our model successfully reaches the goal, even when encountering configurations not explicitly present in the training set. In addition, we compare the performance of the simple regression model with ours for the ablation study, as shown in Fig. 5. Our method outperforms the simple regression model due to the weight of the step control input difference in the loss function (16). This weight appears to identify the critical steps in the planning horizon that are essential for reaching the goal.

Algorithm 2 Constructing dataset

random $\mathcal{O}$
while $d_G(x_t) > 0.6$ m **do**
  sample M trajectories of length N from $\mathcal{N}(\mu_u, \mathbf{\Sigma}_u)$
  evaluate each trajectory using (18)
  choose K trajectories, and update μ_u, $\mathbf{\Sigma}_u$ using (21)
save $[x_0, x_G, \mathcal{O}, \mu_u, \mathbf{\Sigma}_u]$

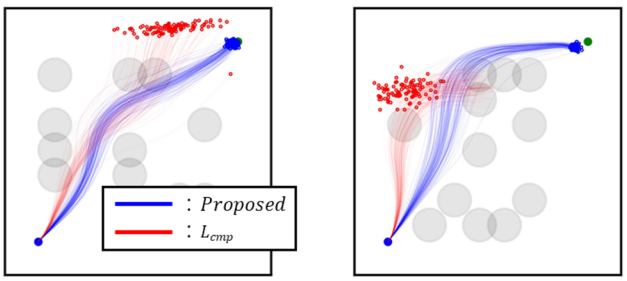


Fig. 5 The ablation study of our work. We compare model trained by $\mathcal{L}_{\text{cmp}} = \mathbb{E}[\| u_{t,\theta}^* - u_t^* \|^2]$ with ours. Our model outperforms the model trained by $\mathcal{L}_{\text{cmp}}$

4.1 Comparison with Trajectory-Wise Denoising

Our method achieves improved efficiency over trajectory-wise denoising methods by aligning the denoising steps with a receding horizon. This design enables dynamic adjustment of trajectory length based on computational constraints, allowing flexible inference under limited resources. The methods proposed in [17–19] generally require a large number of denoising steps to reach convergence (e.g., 256 for [18], 128 for [17], and 50 for [19]) compared to our approach, which makes them difficult to apply as local planners in real-time settings. Among them, [19] requires the fewest denoising steps; however, we exclude it in our comparison due to the lack of publicly available source code. Before comparing ours with [17, 18], we conduct a runtime analysis and find that [17, 18] is not suitable for a fair comparison because of their significant computational overhead. Since [17, 18] refine the entire trajectory at each inference step, these methods cannot leverage intermediate outputs and are thus less adaptable in time-constrained scenarios, as shown in Fig. 6. Therefore, we exclude [17, 18] from our comparison.

Furthermore, our method can be used as an initial guidance for trajectory-wise denoising approaches, significantly reducing the number of iterations required for convergence. To validate this strength, we compare our approach with MPPI [7] and SV-MPC [21], two well-known trajectory-

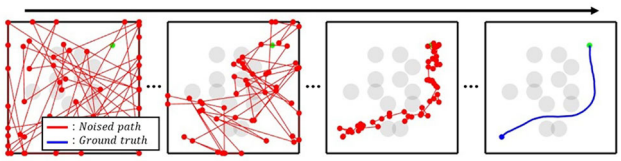


Fig. 6 This figure illustrates the trajectory-wise denoising process using diffusion-based methods [17, 18]. Both [17, 18] require more than 2 s to converge, because the number of iterations needed for path convergence is too large

wise denoising methods, in simulation to generate trajectories toward the goal. Our approach utilizes the network's inference as initial guidance and refines it through MPPI-based denoising. We set the minimum variance to $\mathbf{\Sigma}_w = \text{diag}\left([0.67^2, 1.0^2]\right)$ scaled by 0.04, to avoid vanishing variance that could hinder the convergence of the goal. For MPPI and our proposed method, the sampling count is set to 10,000, whereas for SV-MPC, the sampling count is set to 1000, as SV-MPC can update the trajectory sufficiently with fewer samples. To evaluate performance, we measure the following metrics:

(1) Convergence time (t_c): It captures the total time required for the trajectory to converge, including network inference.
(2) Number of denoising iterations (I_c): It quantifies how many iterations are needed for the planned trajectory to converge to a stable solution.
(3) Goal-reaching success rate (GR): It indicates the proportion of trajectories whose final state lies within a specified distance from the goal.

The trajectory convergence criterion for calculating t_c is based on the similarity distance d_{sim} between collision-free trajectories of the $(k-1)$-th and kth iterations, ensuring that it remains within a predefined threshold, $\epsilon = 2$. The criterion is computed as follows:

$$d_{\text{sim}}\left(\tau_i, \tau_j\right) = \sum_{k=0}^{N-1} d\left(\boldsymbol{x}_k^i, \boldsymbol{x}_k^j\right) \leq \epsilon,$$

where $\boldsymbol{x}_k^i$ is the position of trajectory τ_i at step k. In addition, we regard the goal as reached if the distance between the final state of the planned trajectory and the goal is less than or equal to 0.6 m.

We perform simulations on 100 different maps using the same settings as those used during training. In all simulations, the starting position is fixed at (0.5, 0.5) m with a heading angle of 45 degrees and a velocity of 0 m/s, and the goal position is set to (7.5, 7.5) m. The results are summarized in Table 2, while the path-planning results are presented in Fig. 7.

Table 2 Comparison of methods over 100 maps (mean ± one standard deviation)

	MPPI [7]	SV-MPC [21]	Ours
t_c (s)	1.52 ± 0.63	0.86 ± 0.64	$\mathbf{0.57 \pm 0.28}$
I_c	30 ± 15	17 ± 13	$\mathbf{9 \pm 5}$
GR (%)	46	26	**61**

SV-MPC exhibits minimal changes between iterations, leading to rapid convergence and early termination before reaching the goal. On the other hand, MPPI exhibits larger changes in the determined trajectory between iterations compared to SV-MPC. However, the slow convergence of MPPI causes a longer computation time, because it requires many iterations to reach convergence. Our proposed method sets an appropriate mean and variance of control input sequence as initial guidance, reducing the number of iterations required for convergence while achieving a higher goal-reaching success rate.

4.2 Hardware Configuration and Parameter Setting

Figure 8 illustrates the hardware system architecture. We utilize the DeepRacer platform for our experiments, with its specifications available on the Amazon DeepRacer homepage. The vehicle receives control inputs computed by a dedicated server, which is equipped with an Intel i7-1165G7 CPU, an RTX 2060 GPU, and 8 GB of RAM.

The control input sequence distribution for the MPPI-based planning module is computed using Algorithm 3. The path planner operates with a fixed time budget of 50 ms per iteration, utilizing 10,000 samples for MPPI-based planning. The prediction horizon length is set to 10 to ensure stable path computation at the defined frequency. The network output is used to sample paths, and the cost function (18) is employed in the MPPI framework to determine the DeepRacer's tracking path. The computed trajectory is then transmitted to the PID controller, which updates the control input every 50 ms. The experimental environment consists of a 5×5 m area containing circular obstacles with a radius of 0.34 m, incorporating a safety margin of $d_s = 0.1$ m. The parameters used in the experiment are summarized in Table 3.

Algorithm 3 Computing control input sequence distribution

while task not completed **do**
  get x_k, $\mathcal{O}$ from sensors
  while operation time not exceeded **do**
    $\mu_{u,k}, \Sigma_{u,k} \leftarrow f_{net}\left(x_k, x_G, \mathcal{O}\right)$
    $\Sigma_{u,k} = \max\left(\Sigma_{u,k}, 0.04 \times \Sigma_w\right)$
    $x_k \leftarrow f_{sys}\left(x_k, \mu_{u,k}\right)$
    $k \leftarrow k + 1$
  output μ_u, Σ_u sequence

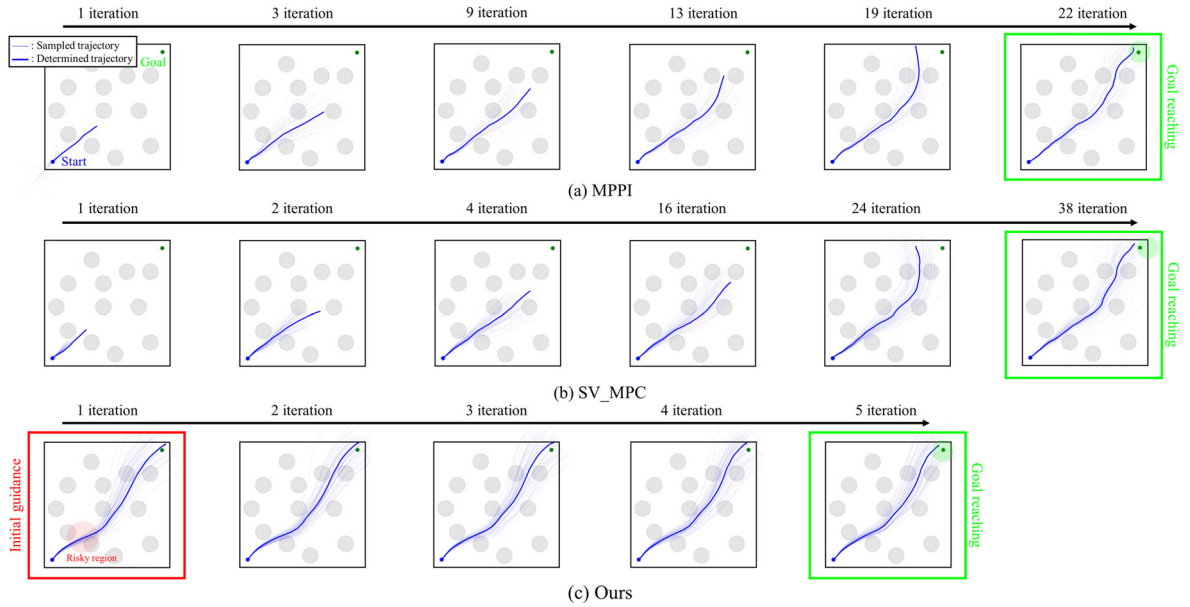


Fig. 7 We compare three approaches: MPPI, SV-MPC, and MPPI augmented with our method. Our method serves as initial guidance for MPPI and demonstrates the benefits of using a learned distribution for initialization. The results indicate that providing well-defined initial guidance enables the planner to achieve the fastest convergence to the goal. Furthermore, when integrated with MPPI, our method enables iterative refinement of unsafe network outputs, thereby mitigating risk

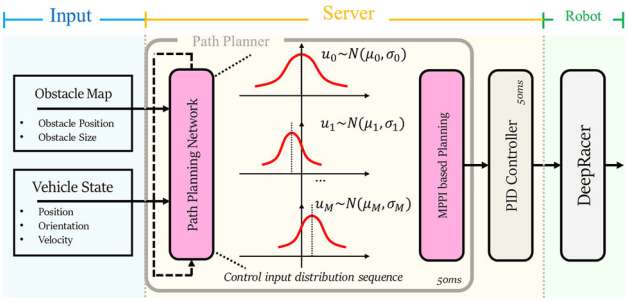


Fig. 8 Architecture of the hardware system. The path-planning network is depicted in Fig. 3. The path planner consists of the path-planning network and MPPI-based planning module. MPPI-based planning module utilizes the network output to sample the paths and determines the path using MPPI

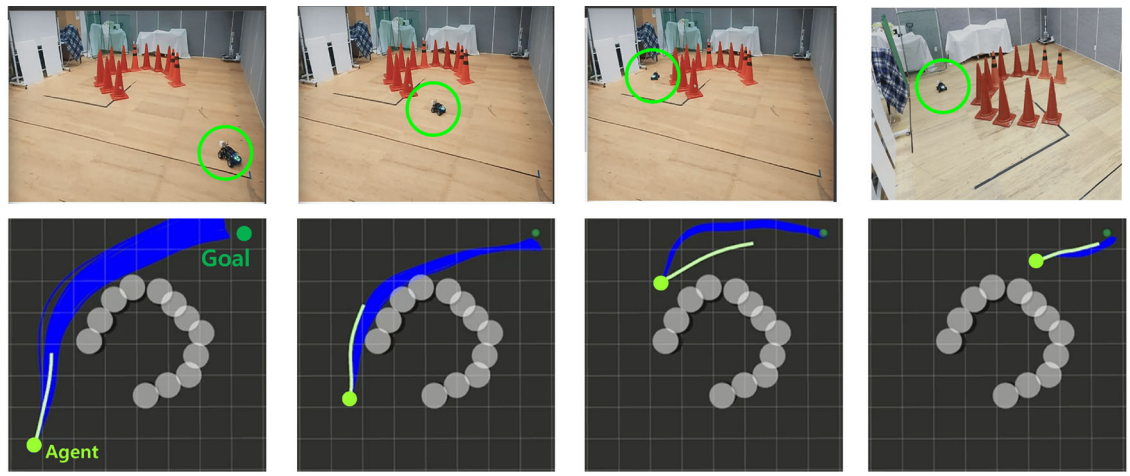
4.3 Experiment on Local Minima and Fast Planning Scenario

Unlike trajectory-wise denoising methods, our approach can utilize intermediate results without requiring iterative refinement until the entire trajectory converges. This allows us to demonstrate the effectiveness of the proposed trajectory-stepwise denoising approach for real-time implementation. To validate our approach, we conduct hardware experiments in two environments: one where the agent can easily become trapped in a local minimum and another with randomly placed obstacles requiring fast planning.

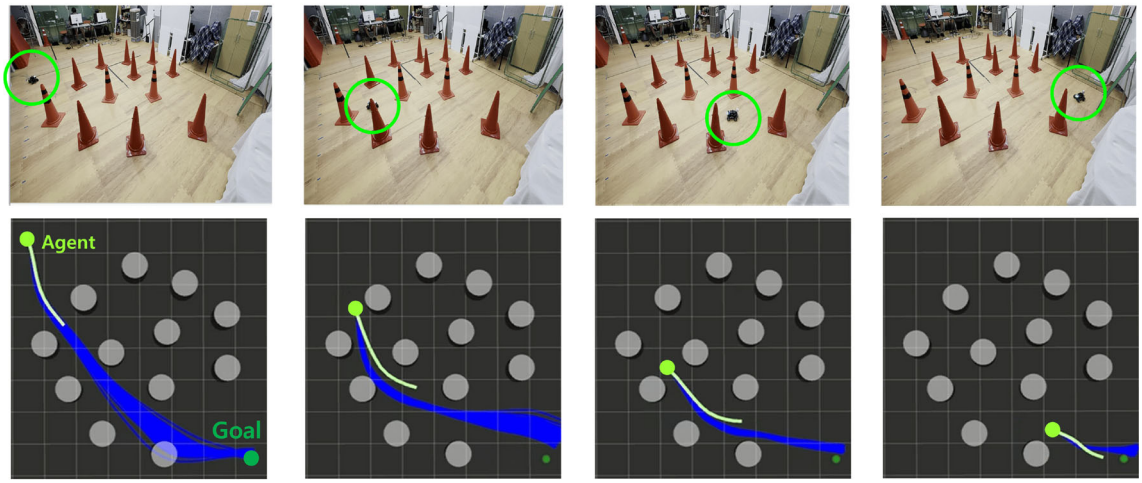
These environments are not included in the training data. In the environment where the agent is at risk of getting

Table 3 Parameter settings for experiment

Parameter	Description	Value	Unit
N	Horizon length	10	·
N_s	Sampling count for MPPI	10,000	·
Δt	Time step interval	0.2	s
d_s	Safety margin	0.1	m
r_{obs}	Obstacle radius	0.34	m
$a_{\max}$	Maximum acceleration	1	m/s^2
$a_{\min}$	Minimum acceleration	-1	m/s^2
$r_{\max}$	Maximum yaw rate	1.2	rad/s
$r_{\min}$	Minimum yaw rate	-1.2	rad/s
Σ_w	Exploration policy covariance	diag([0.67^2, 1.0^2])	·



(a) Scenario including local minima



(b) Scenario requiring fast planning

Fig. 9 Result of the hardware experiment. The blue curve is guidance and the green is the determined path. The prediction horizon length of the guidance is 50, and the prediction horizon length for the determined path is 10. The agent starts at (0.3, 0.3) position with zero velocity and 45 degrees heading angle

trapped in a local minimum, our proposed network successfully navigated to the goal without collisions. In contrast, unguided MPPI with a prediction horizon length of 10 failed to reach the goal when solving the optimal control problem using the cost function (18). Fast local path planning is crucial in environments with randomly placed obstacles. Since our proposed method utilizes initial guidance, it can rapidly find a collision-free path without requiring additional iterations. Figure 9 presents the experimental results, showing that the agent successfully avoids collisions and local minima, re-plans efficiently, and reaches the goal.

5 Conclusion

In this paper, we propose a novel learning-based path-planning network utilizing CFM. Our approach enables more efficient trajectory generation by defining the NF as the trajectory-stepwise denoising over the receding time horizon. This method allows the use of intermediate network outputs, enhancing the flexibility of the trajectory generation considering computation resources. We demonstrate the practicality of our method through comprehensive testing, including a real-world hardware demonstration. In complex environments, multiple feasible paths may exist to reach the goal state. However, our current approach assumes a unimodal control input distribution, which limits its ability to capture the multimodality of such paths. In future work, we plan to extend our approach by leveraging the multimodality of NF to enable multitrajectory planning in complex environments.

Author Contributions Yongjae Lim designed the method, ran the simulations and experiments, and wrote the paper. Yeonsoo Park contributed to the network training. Jeongtae Huh assisted with the experiments. Youngsoo Han helped review and refine the manuscript. H. Jin Kim provided guidance on the research direction.

Funding Open Access funding enabled and organized by Seoul National University. This work was supported by the IITP (Institute of Information & Communications Technology Planning & Evaluation)-ITRC (Information Technology Research Center) grant funded by the Korea government (Ministry of Science and ICT) (IITP-2025-RS-2024-00437268).

Data availability No external datasets were generated or analyzed during the current study.

Declarations

Conflict of interest H. Jin Kim is a Senior Editor of International Journal of Control, Automation, and Systems. Senior Editor status has no bearing on editorial consideration. The authors declare that there is no competing financial interest or personal relationship that could have appeared to influence the work reported in this paper.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Yi, B., Gottschling, S., Ferdinand, J., Simm, N., Bonarens, F., Stiller, C.: Real time integrated vehicle dynamics control and trajectory planning with mpc for critical maneuvers. In: 2016 IEEE Intelligent Vehicles Symposium (IV), pp. 584–589 (2016)
2. Ji, J., Khajepour, A., Melek, W.W., Huang, Y.: Path planning and tracking for vehicle collision avoidance based on model predictive control with multiconstraints. IEEE Trans. Veh. Technol. **66**(2), 952–964 (2017)
3. Kim, J.-C., Pae, D.-S., Lim, M.-T.: Obstacle avoidance path planning based on output constrained model predictive control. Int. J. Control Autom. Syst. **17**(11), 2850–2861 (2019)
4. Li, J., Ran, M., Wang, H., Xie, L.: Mpc-based unified trajectory planning and tracking control approach for automated guided vehicles. In: 2019 IEEE 15th International Conference on Control and Automation (ICCA), pp. 374–380 (2019)
5. Ko, C., Han, S., Choi, M., Kim, K.-S.: Integrated path planning and tracking control of autonomous vehicle for collision avoidance based on model predictive control and potential field. In: 2020 20th International Conference on Control, Automation and Systems (ICCAS), pp. 956–961 (2020)
6. Zhu, G., Jie, H., Zheng, Z., Hong, W.: Mpc-based lateral motion control for autonomous vehicles through serially cascaded discretization method considering path preview. Int. J. Control Autom. Syst. **22**(5), 1666–1679 (2024)
7. Williams, G., Drews, P., Goldfain, B., Rehg, J.M., Theodorou, E.A.: Information-theoretic model predictive control: theory and applications to autonomous driving. IEEE Trans. Rob. **34**(6), 1603–1622 (2018)
8. Yin, J., Zhang, Z., Theodorou, E., Tsiotras, P.: Trajectory distribution control for model predictive path integral control using covariance steering. In: 2022 International Conference on Robotics and Automation (ICRA), pp. 1478–1484 (2022)
9. Balci, I.M., Bakolas, E., Vlahov, B., Theodorou, E.A.: Constrained covariance steering based tube-mppi. In: 2022 American Control Conference (ACC), pp. 4197–4202 (2022)
10. Higgins, J., Mohammad, N., Bezzo, N.: A model predictive path integral method for fast, proactive, and uncertainty-aware uav planning in cluttered environments. In: 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 830–837 (2023)
11. Honda, K., Akai, N., Suzuki, K., Aoki, M., Hosogaya, H., Okuda, H., Suzuki, T.: Stein variational guided model predictive path integral control: Proposal and experiments with fast maneuvering vehicles (2023). arXiv:2309.11040
12. Theodorou, E.A., Todorov, E.: Relative entropy and free energy dualities: connections to path integral and kl control. In: 2012 IEEE 51st IEEE Conference on Decision and Control (CDC), pp. 1466–1473 (2012)
13. Okada, M., Taniguchi, T.: Variational inference mpc for bayesian model-based reinforcement learning. In: Kaelbling, L.P., Kragic, D., Sugiura, K. (eds.) Proceedings of the Conference on Robot Learning, ser. Proceedings of Machine Learning Research, vol. 100. PMLR, 30 Oct–01 Nov, pp. 258–272 (2020). [Online]. https://proceedings.mlr.press/v100/okada20a.html
14. Yin, J., Dawson, C., Fan, C., Tsiotras, P.: Shield model predictive path integral: a computationally efficient robust mpc method using control barrier functions. IEEE Rob. Autom. Lett. **8**(11), 7106–7113 (2023)
15. Kusumoto, R., Palmieri, L., Spies, M., Csiszar, A., Arras, K.O.: Informed information theoretic model predictive control. In: 2019 International Conference on Robotics and Automation (ICRA), pp. 2047–2053 (2019)
16. Qu, Y., Chu, H., Gao, S., Guan, J., Yan, H., Xiao, L., Li, S.E., Duan, J.: Rl-driven mppi: accelerating online control laws calculation with offline policy. IEEE Trans. Intell. Veh. **9**(2), 3605–3616 (2024)
17. Janner, M., Du, Y., Tenenbaum, J.B., Levine, S.: Planning with diffusion for flexible behavior synthesis (2022) [Online]. arXiv:2205.09991
18. Xiao, W., Wang, T.-H., Gan, C., Rus, D.: Safediffuser: safe planning with diffusion probabilistic models (2023). [Online]. arXiv:2306.00148
19. Power, T., Berenson, D.: Learning a generalizable trajectory sampling distribution for model predictive control. IEEE Trans. Rob. **40**, 2111–2127 (2024)
20. Lipman, Y., Chen, R.T.Q., Ben-Hamu, H., Nickel, M., Le, M.: Flow matching for generative modeling (2023). [Online]. arXiv:2210.02747
21. Lambert, A., Fishman, A., Fox, D., Boots, B., Ramos, F.: Stein variational model predictive control (2020). arXiv:2011.07641
22. Hansen, N., Wang, X., Su, H.: Temporal difference learning for model predictive control (2022)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Yongjae Lim received the B.S. degree in Autonomous engineering from Hanyang University, South Korea, in 2023. He is currently pursuing the Ph.D. degree in mechanical engineering at Seoul National University, Seoul, South Korea. His research interests include autonomous vehicle control and path planning of autonomous vehicles.



Yeonsoo Park received the B.S. degree in Electrical & Electronic Engineering from Yonsei University, South Korea, in 2024. She is currently pursuing the M.S. degree in Artificial Intelligence at Seoul National University, South Korea. Her research interests include Generative AI, Spatial AI and 3D Vision.



JeongTae Huh received the B.S. degree in mechanical engineering from Yonsei University, Seoul, South Korea, in 2023. He is currently pursuing M.S. degree in Department of Future Automative Mobility. His current research interests include path planning and system control.



Youngsoo Han received the B.S degree in the Department of Mechanical and Aerospace Engineering, Seoul National University, Seoul, South Korea, in 2020. He is currently working toward the integrated M.S./Ph.D. degree with the Aerospace Engineering, Seoul National University, Seoul, South Korea. His research interests include visual SLAM, camera and LiDAR odometry, point cloud registration, and sensor fusion.



H. Jin Kim received the B.S. degree in mechanical engineering from the Korean Advanced Institute of Technology, Daejeon, South Korea, in 1995, and the M.S. and Ph.D. degrees in mechanical engineering from the University of California at Berkeley, Berkeley, CA, USA, in 1999 and 2001, respectively. From 2002 to 2004, she was a Post-Doctoral Researcher with the Department of Electrical Engineering and Computer Science, University of California, Berkeley. In 2004, she joined the School of Mechanical and Aerospace Engineering, Seoul National University, Seoul, South Korea, where she is currently a Professor. Her research interests include navigation and motion planning of autonomous robotic systems.